EECS 348: Assignment 4

Analyze the two GenAI programs. The analysis must include:
a. Name of two GenAI programs
b. How you accessed them (e.g., Google query, CoPilot on my computer)
ChatGPT (https://chatgpt.com/) and Gemini (https://gemini.google.com/app)

c. Prompt used. (You must use the same prompt for both GenAIs.)
Prompt:
Write a single-file C++ program (Assignment4.cpp) that solves Sudoku puzzles
using a recursive, depth-first backtracking search. Use an object-oriented
design (a class that holds the grid and has methods), not free functions.

Input:
- Five text files: puzzle1.txt through puzzle5.txt, in the same folder as the program.
- Each file has 9 lines of 9 space-separated symbols: digits 1-9, or "_" for a blank.

Rules:
- Every row, every column, and every 3x3 block must contain digits 1-9 exactly once.

Algorithm:
- Find the next empty cell and compute its candidate digits (not already in its row, column, or block).
- If there are no candidates, backtrack.
- Otherwise try each candidate: place it, recurse, then undo it.
- Do NOT stop at the first solution. Find and print ALL solutions.

Output, for each puzzle file:
1. The file name (e.g., puzzle1.txt)
2. The original puzzle as read from the file
3. Each solution as a 9x9 grid, followed by "Solutions found: N"
4. If none exist, print "No solution found"

Requirements:
- Must compile with g++ on a Linux server using a standard C++ (C++11 or later) compiler.
- Use only the standard library.
- Read the five files by name and process them in order.

d. Generated code from both GenAIs.
ChatGPT Output:

```
#include <iostream>
#include <fstream>
#include <string>
```

```
#include <vector>

using namespace std;

class Sudoku {
private:
    int grid[9][9];
    int solutionCount;

    // Check whether a digit can be placed at (row, col).
    bool isValid(int row, int col, int num) {
        // Check the row and column.
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == num ||
                grid[i][col] == num) {
                return false;
            }
        }

        // Check the 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if (grid[r][c] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    // Find the next empty cell.
    bool findEmptyCell(int &row, int &col) {
        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {
                if (grid[row][col] == 0) {
                    return true;
                }
            }
        }
```

```
        return false;
    }

    // Print the current grid.
    void printGrid() {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (col > 0) {
                    cout << " ";
                }
                cout << grid[row][col];
            }
            cout << endl;
        }
    }

    // Recursively find and print every solution.
    void solveRecursive() {
        int row, col;

        // No empty cells means a complete solution was found.
        if (!findEmptyCell(row, col)) {
            solutionCount++;
            cout << "\nSolution " << solutionCount << ":\n";
            printGrid();
            return;
        }

        // Try every candidate digit.
        for (int num = 1; num <= 9; num++) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;

                // Continue searching for solutions.
                solveRecursive();

                // Undo the choice to explore other candidates.
                grid[row][col] = 0;
            }
        }
    }

public:
    Sudoku() {
```

```
    solutionCount = 0;

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            grid[row][col] = 0;
        }
    }
}

// Read a puzzle from a file. "_" represents an empty cell.
bool loadPuzzle(const string &filename) {
    ifstream file(filename.c_str());

    if (!file.is_open()) {
        cerr << "Error: Could not open " << filename << endl;
        return false;
    }

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            string symbol;

            if (!(file >> symbol)) {
                cerr << "Error: Invalid puzzle format in "
                     << filename << endl;
                return false;
            }

            if (symbol == "_") {
                grid[row][col] = 0;
            } else if (symbol.size() == 1 &&
                       symbol[0] >= '1' &&
                       symbol[0] <= '9') {
                grid[row][col] = symbol[0] - '0';
            } else {
                cerr << "Error: Invalid symbol in "
                     << filename << endl;
                return false;
            }
        }
    }

    return true;
}
```

```
    // Print the puzzle as originally loaded.
    void printOriginal() {
        printGrid();
    }

    // Find and print all solutions.
    void solve() {
        solutionCount = 0;
        solveRecursive();

        if (solutionCount == 0) {
            cout << "No solution found" << endl;
        } else {
            cout << "Solutions found: " << solutionCount << endl;
        }
    }
};

int main() {
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";

        cout << "\n========================================\n";
        cout << filename << endl;
        cout << "========================================\n";

        Sudoku sudoku;

        if (!sudoku.loadPuzzle(filename)) {
            continue;
        }

        cout << "\nOriginal puzzle:\n";
        sudoku.printOriginal();

        cout << "\nSolving puzzle...\n";
        sudoku.solve();
    }

    return 0;
}
```

Gemini Output:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
    int grid[9][9];
    int solutionCount;

    bool isValid(int row, int col, int num) const {
        for (int c = 0; c < 9; ++c) {
            if (grid[row][c] == num) {
                return false;
            }
        }

        for (int r = 0; r < 9; ++r) {
            if (grid[r][col] == num) {
                return false;
            }
        }

        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (grid[startRow + r][startCol + c] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    std::vector<int> getCandidates(int row, int col) const {
        std::vector<int> candidates;
        for (int num = 1; num <= 9; ++num) {
            if (isValid(row, col, num)) {
                candidates.push_back(num);
            }
        }
```

```
        return candidates;
    }

    bool findNextEmpty(int &row, int &col) const {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (grid[r][c] == 0) {
                    row = r;
                    col = c;
                    return true;
                }
            }
        }
        return false;
    }

    void solveRecursive() {
        int row = -1;
        int col = -1;

        if (!findNextEmpty(row, col)) {
            solutionCount++;
            std::cout << "\nSolution " << solutionCount << ":\n";
            printGrid();
            return;
        }

        std::vector<int> candidates = getCandidates(row, col);

        if (candidates.empty()) {
            return;
        }

        for (int val : candidates) {
            grid[row][col] = val;
            solveRecursive();
            grid[row][col] = 0;
        }
    }

public:
    SudokuSolver() : solutionCount(0) {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
```

```
            grid[r][c] = 0;
        }
    }
}

bool loadFromFile(const std::string &filename) {
    std::ifstream inFile(filename);
    if (!inFile.is_open()) {
        return false;
    }

    solutionCount = 0;
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            std::string token;
            if (!(inFile >> token)) {
                inFile.close();
                return false;
            }
            if (token == "_") {
                grid[r][c] = 0;
            } else {
                grid[r][c] = token[0] - '0';
            }
        }
    }
    inFile.close();
    return true;
}

void printGrid() const {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            if (grid[r][c] == 0) {
                std::cout << "_ ";
            } else {
                std::cout << grid[r][c] << " ";
            }
        }
        std::cout << "\n";
    }
}

void solve() {
```

```
        solveRecursive();
        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        } else {
            std::cout << "Solutions found: " << solutionCount << "\n";
        }
    }
};

int main() {
    const std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const auto &filename : puzzleFiles) {
        std::cout << "========================================\n";
        std::cout << filename << "\n";
        std::cout << "========================================\n";

        SudokuSolver solver;
        if (!solver.loadFromFile(filename)) {
            std::cerr << "Error: Could not open or read " << filename << "\n\n";
            continue;
        }

        std::cout << "Original Puzzle:\n";
        solver.printGrid();

        std::cout << "\nSolving...\n";
        solver.solve();
        std::cout << "\n";
    }

    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

The ChatGPT program is correct for the core task: recursive backtracking places a digit, recurses, and undoes it, so it finds every solution and reports either the count or “No solution found”. It validates file symbols and reports read errors, but it never checks whether the given clues already conflict, and it prints blanks as 0 instead of the file’s underscore. In general, execution time is exponential in the worst case. Each call rescans from the first cell and tests each digit against a row, column, and block, but it allocates no extra containers, so per-call overhead is low. Space is O(1) beyond the 81-cell grid and a recursion depth of at most 81 frames. Maintainability is good: one class with small private methods and clear names. Minor weaknesses are using namespace std, an unused <vector> include, and repeated magic numbers (9 and 3).

The Gemini program is also correct for the core task: it uses the same place, recurse, undo pattern, finds all solutions, and prints blanks as underscores, matching the input files. It does not validate file symbols (token[0] - '0' accepts bad input) and does not check for conflicting clues. In general, its search order and exponential worst case match ChatGPT’s, but it builds a std::vector of candidates in every recursive call, adding heap-allocation overhead, so it is likely somewhat slower. Space is still small: the 81-cell grid, a depth of at most 81 frames, and a vector of at most nine integers per frame. Maintainability is slightly stronger: std:: prefixes, const methods, a separate getCandidates helper, and range-based loops. It has a redundant empty-candidate check and magic numbers.

f. Select one to use as the basis for this assignment and justify why you picked it.

I selected Gemini’s program as the basis. Both programs use a recursive, object-oriented backtracking search and find every solution. Gemini won on maintainability and output: it uses std:: prefixes instead of using namespace std, marks read-only methods const, splits candidate logic into its own getCandidates method, and prints blanks as _, matching the input files, while ChatGPT prints 0. Gemini’s one weakness is that it builds a std::vector of candidates in every recursive call, which I could remove to improve time and space.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

I improved Gemini’s code in four ways.

- Correctness: I added symbol validation so invalid file characters are rejected, and a clue check that prints “No solution found” immediately when given clues conflict.
- Execution time: I removed the per-call candidate vector, so each recursive call no longer allocates heap memory.
- Space complexity: each recursion frame no longer holds a vector, leaving only the 81-cell grid and at most 81 stack frames.
- Maintainability: I deleted the redundant empty-candidate check, since the digit loop already backtracks naturally.